



EnergyGuard Mini

Privacy-Preserving Smart Home Energy Intelligence

A Complete Beginner-Friendly Build Guide

Covers: Python · Pandas · NumPy · Random Forest · Isolation Forest · Federated Learning · SHAP · FastAPI · Streamlit

Learning Project for GridGuard FYP Preparation

Table of Contents

Part 0	Before You Write Any Code — Essential Theory	 3
Part 1	Environment Setup	 5
Part 2	Data — The Foundation of Everything	 7
Part 3	Machine Learning with Random Forest	 11
Part 4	Anomaly Detection with Isolation Forest	 15
Part 5	Virtual Homes — Non-IID Distributed Data	 18
Part 6	Federated Learning — The Core Concept	 21
Part 7	Explainable AI with SHAP	 29
Part 8	Agentic AI — The Decision Layer	 33
Part 9	FastAPI — Turning AI into a Service	 36
Part 10	Streamlit Dashboard	 41
Part 11	Putting It All Together	 47
Part 12	Testing & Evaluation	 49

Part 0: Understanding the Big Picture

Before touching any code, you must understand WHY each piece exists. This section explains every major concept in simple language.

0.1 What is Machine Learning?

Machine Learning (ML) is teaching a computer to find patterns in data without explicitly programming every rule. Instead of writing IF-THEN rules, you show the computer thousands of examples and it learns the rules automatically.

💡 *Real-world analogy: Instead of writing "if person is tall AND young AND male → basketball player", you show 10,000 photos of basketball players and let the computer figure out the pattern itself.*

Two types of ML we use:

- Supervised Learning — You give examples WITH correct answers. "Given these features, predict this value." This is how we do energy forecasting.
- Unsupervised Learning — You give data WITHOUT correct answers. The model finds hidden patterns. This is how anomaly detection works.

0.2 What is a Feature and a Target?

These are the two most important words in ML:

Term	Meaning	Example in our project
Feature	Input data — what you feed into the model	Previous consumption, hour, day of week
Target	Output — what you want to predict	Next energy consumption value
Training	Teaching the model using historical data	First 80% of our dataset
Testing	Evaluating how well the model learned	Last 20% of our dataset

0.3 What is Federated Learning?

Federated Learning (FL) is a way to train ML models without sharing raw data. Each location trains a model locally and only shares the model parameters (the learned numbers), not the actual data.

Without FL (Centralized): Home A data → Home B data → Central Server → trains ONE model
Home C data → Problem: Privacy! All raw data goes to one place.
With Federated Learning: Home A trains locally → sends only "model weights" → Home B trains locally → sends only "model weights" → Server combines weights
Home C trains locally → sends only "model weights" → Benefit: Raw data NEVER leaves each home.

0.4 What is SHAP / Explainable AI?

SHAP (SHapley Additive exPlanations) answers the question: "Why did the model make this prediction?" It tells you how much each feature contributed to a specific prediction.

 *Example: The model predicted 4.8 kWh. SHAP tells you: "Previous consumption contributed +1.2, rolling average contributed +0.8, and hour of day contributed +0.3." Now you understand WHY.*

0.5 What is an AI Agent?

An AI agent is a system that: observes the environment (reads data), makes a decision (runs logic), and takes an action (outputs a recommendation). In our project, the agent reads forecasts and anomaly scores and decides what action to recommend.

0.6 What is an API?

An API (Application Programming Interface) is a way for different software systems to talk to each other over a network. FastAPI lets us turn our Python ML code into a web service that any other program can call.

```
Without API: Python script runs, prints result, done. With API: Python ML
model runs as a server. Other apps send requests → get
predictions back. URL: http://localhost:8000/forecast/home-a
Response: {"predicted_consumption": 3.82}
```

Part 1: Setting Up Your Environment

1.1 What You Need to Install

We need Python (the programming language) and several libraries. A library is pre-written code that someone else built, so we don't start from zero.

Step 1: Install Python

Download Python 3.10 or higher from python.org. During installation on Windows, check "Add Python to PATH".

```
# Verify Python is installed:python --version# Should show: Python 3.10.x or higher
```

Step 2: Create a Project Folder

```
# Open your terminal/command promptmkdir energyguard_minicd energyguard_mini
```

Step 3: Create a Virtual Environment

A virtual environment is an isolated Python setup just for this project. This prevents conflicts with other projects.

```
# Create virtual environmentpython -m venv venv# Activate it
(Windows):venv\Scripts\activate# Activate it (Mac/Linux):source
venv/bin/activate# You should see (venv) at the start of your terminal prompt
```

Step 4: Install All Libraries

```
pip install numpy pandas matplotlib scikit-learn shap fastapi uvicorn streamlit
flwr
```

1.2 Your Project Structure

Create this folder structure. Each file has a specific job.

```
energyguard_mini/| | data/| | energy_data.csv | | ← raw dataset goes
here| | homes/| | | home_a.csv | | ← data for Home A| | |
home_b.csv | | ← data for Home B| | | home_c.csv | | ←
data for Home C| | | models/| | | model_a.pkl | | ← trained model for
Home A| | | model_b.pkl | | ← trained model for Home B| | |
model_c.pkl | | ← trained model for Home C| | | 01_data_preparation.py
← Part 2| | | 02_ml_forecasting.py | | ← Part 3| | | 03_anomaly_detection.py
← Part 4| | | 04_virtual_homes.py | | ← Part 5| | | 05_federated_learning.py
← Part 6| | | 06_shap_explainability.py | | ← Part 7| | | 07_energy_agent.py
← Part 8| | | 08_api.py | | ← Part 9| | | 09_dashboard.py
← Part 10
```

 Create these folders now: `mkdir data mkdir homes mkdir models`

Part 2: Understanding and Preparing Data

2.1 What Data Do We Use?

We use energy consumption data. Each row represents one measurement at a specific time. We'll generate realistic synthetic data so you don't need to download anything.

What our data looks like:

timestamp	consumption_kwh	hour	day_of_week
2024-01-01 00:00	0.8	0	Monday
2024-01-01 00:15	0.7	0	Monday
2024-01-01 08:00	1.5	8	Monday
2024-01-01 12:00	2.1	12	Monday

2.2 Understanding Pandas

Pandas is the most important library for data work. It gives us the DataFrame — a table-like structure for data manipulation.

Key Pandas concepts:

Concept	What it means	Example
DataFrame	A table with rows and columns	<code>df = pd.read_csv("data.csv")</code>
Series	A single column of data	<code>df["consumption_kwh"]</code>
Index	Row labels (usually numbers)	<code>df.iloc[0]</code> ← first row
loc	Select rows by label	<code>df.loc[df["hour"] == 8]</code>
groupby	Group rows and aggregate	<code>df.groupby("hour").mean()</code>

2.3 Complete Data Preparation Code

Create the file `01_data_preparation.py` and add every line below. Read each comment carefully — it explains exactly what the code does.

```
# 01_data_preparation.py# —————# This file does ONE job: create realistic energy# consumption data and save it to CSV files.# —————import numpy as np# For mathematicsimport pandas as pd          # For data tablesimport matplotlib.pyplot as plt  # For chartsimport os                                # For folder creation# — Set a random seed —————# This makes our random data the same every run.# Without this, each run gives different numbers.np.random.seed(42)# — Create time range —————# We create timestamps every 15 minutes for 60 days.# pd.date_range creates a sequence of dates.timestamps = pd.date_range(    start='2024-01-01',    # Start date    end='2024-03-01',    # End date    freq='15min'    # Every 15
```

```

minutes)print(f"Total timestamps: {len(timestamps)}")# This will print around
5760 timestamps# — Understand energy patterns —————# Real homes
consume energy in patterns:# - Low at night (people sleeping)# - High in
morning (breakfast, shower)# - Medium in afternoon# - High in evening
(cooking, TV)def generate_home_data(timestamps, base_load, noise_level, name):
    """    Generate realistic energy consumption for ONE home.    Parameters:
- timestamps : when the measurements happen - base_load : average
consumption (kWh) - noise_level : how variable the consumption is - name
: home name (for printing)    """    data = [] # We'll collect each row
here    for ts in timestamps:        hour = ts.hour        # 0-23
day = ts.dayofweek        # 0=Monday ... 6=Sunday        # — Hourly
pattern —————        # This creates a realistic daily curve.
# np.sin creates a wave shape.        # The formula creates: low at night,
peaks at 8am and 7pm.        hourly_pattern = (        0.5 * np.sin(2 *
np.pi * (hour - 6) / 24)        # morning peak        + 0.3 * np.sin(2 * np.pi *
(hour - 19) / 12)        # evening peak        + 1.0
# baseline offset        )        # — Weekend pattern
—————        # Weekends have different consumption patterns
# day >= 5 means Saturday or Sunday        weekend_factor = 1.1 if day >= 5
else 1.0        # — Calculate consumption —————        #
base_load : how much this home typically uses        # hourly_pattern :
multiplied by time-of-day factor        # weekend_factor : multiplied by day
factor        # np.random.normal : adds realistic random variation
consumption = (        base_load        * hourly_pattern        *
weekend_factor        + np.random.normal(0, noise_level) # random noise
)        # — Ensure positive value —————        # Energy can't
be negative!        consumption = max(0.1, consumption)
data.append({        'timestamp': ts,        'consumption_kwh':
round(consumption, 3),        'hour': hour,
'day_of_week': day,        'is_weekend': int(day >= 5),
'month': ts.month,        })    df = pd.DataFrame(data)
print(f"\n{name} data shape: {df.shape}")    print(f"{name} average
consumption: {df['consumption_kwh'].mean():.2f} kWh")    return df# — Generate
data for three homes —————# Each home has different characteristics -
this# creates our Non-IID (non-identical) distribution.print("\n=== Generating
Home Data ===")# Home A: Small family, moderate consumptiondf_a =
generate_home_data( timestamps, base_load=1.5, # moderate base
noise_level=0.15, # low variability name="Home A")# Home B: Large family,
high consumptiondf_b = generate_home_data( timestamps, base_load=3.5,
# high base noise_level=0.35, # more variable name="Home B")# Home C:
Single person, variable scheduleddf_c = generate_home_data( timestamps,
base_load=2.0, # medium base noise_level=0.5, # high variability
name="Home C")# — Inject anomalies —————# We manually add
some "unusual" consumption events.# These simulate faulty appliances, left-on
devices, etc.# We'll use these later to test anomaly detection.def
inject_anomalies(df, n_anomalies=20):    """    Replace n random rows with
unusually high consumption.    Returns the DataFrame with anomaly labels.
    """    df = df.copy()    # Add a label column (0=normal, 1=anomaly)
df['is_anomaly'] = 0    # Pick random positions to inject anomalies
anomaly_indices = np.random.choice( df.index[100:-100], # avoid first
and last rows size=n_anomalies, replace=False )    # Set
those positions to very high consumption    for idx in anomaly_indices:
df.at[idx, 'consumption_kwh'] = np.random.uniform(8, 15)    df.at[idx,
'is_anomaly'] = 1    print(f" Injected {n_anomalies} anomalies")    return
dfprint("\n=== Injecting Anomalies ===")df_a = inject_anomalies(df_a)df_b =
inject_anomalies(df_b)df_c = inject_anomalies(df_c)# — Feature Engineering
—————# Raw data isn't enough for ML. We create additional#
features that help the model make better predictions.def add_features(df):
    """    Add time-based and lag features.    Lag features: the value from N
steps ago.    Rolling features: averages over recent windows.    These give the
model context about recent history.    """    df = df.copy()    df =
df.sort_values('timestamp').reset_index(drop=True)    # Lag features (what
was consumption N periods ago)    df['lag_1'] = df['consumption_kwh'].shift(1)
# 15 min ago    df['lag_4'] = df['consumption_kwh'].shift(4)    # 1 hour ago
df['lag_96'] = df['consumption_kwh'].shift(96)    # 24 hours ago    # Rolling
average (average over last N periods)    df['rolling_mean_4'] =
df['consumption_kwh'].rolling(4).mean()    df['rolling_mean_96'] =
df['consumption_kwh'].rolling(96).mean()    # Rolling standard deviation

```

```

(measures variability)    df['rolling_std_4'] =
df['consumption_kwh'].rolling(4).std()          # Drop rows with NaN (lag creates
NaN at the start)    df = df.dropna().reset_index(drop=True)    return
dfprint("\n=== Adding Features ===")df_a = add_features(df_a)df_b =
add_features(df_b)df_c = add_features(df_c)print(f"\nFeatures after
engineering:")print(df_a.columns.tolist())# — Save to CSV
os.makedirs('homes',
exist_ok=True)df_a.to_csv('homes/home_a.csv',
index=False)df_b.to_csv('homes/home_b.csv',
index=False)df_c.to_csv('homes/home_c.csv', index=False)print("\n=== Saved CSV
Files ===")print(" homes/home_a.csv")print(" homes/home_b.csv")print("
homes/home_c.csv")# — Visualize the data fig, axes =
plt.subplots(3, 1, figsize=(14, 10))for ax, df, name, color in zip(axes,
[df_a, df_b, df_c], ['Home A', 'Home B', 'Home C'], ['steelblue',
'darkorange', 'forestgreen']): sample = df.head(96) # First day
ax.plot(sample.index, sample['consumption_kwh'], color=color,
linewidth=1.2, label='Consumption') anomalies = sample[sample['is_anomaly']
== 1] ax.scatter(anomalies.index, anomalies['consumption_kwh'],
color='red', s=80, zorder=5, label='Anomaly') ax.set_title(f'{name} – First
24 Hours', fontsize=13) ax.set_ylabel('kWh') ax.legend() ax.grid(True,
alpha=0.3)plt.tight_layout()plt.savefig('data_overview.png', dpi=150,
bbox_inches='tight')plt.show()print("\nChart saved:
data_overview.png")print("\n✅ Data preparation complete!")

```

How to run:

```
python 01_data_preparation.py
```

✅ Expected output: Three CSV files in homes/ folder, a chart showing consumption patterns, and statistics printed in the terminal.

Part 3: Building the Forecasting Model

3.1 What is Random Forest?

A Random Forest is a collection of many Decision Trees that work together. Each tree makes a prediction, and the forest takes the majority vote (for classification) or the average (for regression).

```
Decision Tree (one tree): Is consumption > 2 kWh? |— YES: Is hour > 18? |
|— YES: Predict 3.5 |— NO: Predict 2.8 |— NO: Is it weekend?
|— YES: Predict 1.8 |— NO: Predict 1.2
Random Forest (many
trees): Tree 1 predicts: 2.9 Tree 2 predicts: 3.1 Tree 3 predicts: 3.0 Tree 4
predicts: 2.8 Tree 5 predicts: 3.0 Average → Final prediction: 2.96
```

3.2 What is MAE and RMSE?

We need to measure how wrong our model is. We use two metrics:

Metric	Formula	Meaning
MAE	$\text{average}(\text{predicted} - \text{actual})$	Average error in same units as data
RMSE	$\sqrt{\text{average}((\text{predicted} - \text{actual})^2)}$	Punishes large errors more heavily

Lower values = better model. MAE of 0.3 means on average our prediction is 0.3 kWh off.

3.3 What is Overfitting?

Overfitting happens when a model learns the training data too well — it memorizes noise instead of patterns. It performs great on training data but badly on new data. This is why we always test on data the model has NEVER seen.

⚠️ Key rule: NEVER use test data during training. NEVER evaluate on training data. Always keep them completely separate.

3.4 Complete Forecasting Code

```
# 02_ml_forecasting.py# —————# Trains a
Random Forest to predict next energy# consumption value using historical
features.# —————import pandas as
pdimport numpy as npimport matplotlib.pyplot as pltfrom sklearn.ensemble import
RandomForestRegressorfrom sklearn.metrics import mean_absolute_error,
mean_squared_errorimport pickle # For saving/loading modelsimport
osos.makedirs('models', exist_ok=True)# — Define which columns are features
—————# These are the inputs we give to the model.FEATURE_COLUMNS = [
'lag_1', # Consumption 15 min ago 'lag_4', # Consumption
1 hour ago 'lag_96', # Consumption 24 hours ago
'rolling_mean_4', # Average over last hour 'rolling_mean_96', # Average
over last day 'rolling_std_4', # Variability over last hour 'hour',
# Hour of day (0-23) 'day_of_week', # Day (0=Mon ... 6=Sun)
'is_weekend', # 1 if weekend, 0 if weekday 'month', # Month
(1-12)]# The target is what we want to predictTARGET_COLUMN =
'consumption_kwh'def train_model(home_name, filepath): """ Train a Random
Forest for one home. Returns the model and its evaluation metrics.
""" print(f"\n{'='*50}") print(f"Training model for {home_name}")
print(f"{'='*50}") # — Load data ————— df =
```

```

pd.read_csv(filepath, parse_dates=['timestamp'])    print(f"Data loaded:
{len(df)} rows")    # — Separate features and target ————— X =
df[FEATURE_COLUMNS].values    # Feature matrix (2D array)    y =
df[TARGET_COLUMN].values    # Target vector (1D array)    print(f"Feature
matrix shape: {X.shape}")    # (rows, features)    print(f"Target vector shape:
{y.shape}")    # (rows,)    # — Train/Test Split —————
# IMPORTANT: We use TIME-BASED split, not random split.    # Why? Because
energy data is sequential.    # Using random split would let the model "see the
future"    # which is data leakage.    #    # 80% of data for training, 20% for
testing.    split_idx = int(len(df) * 0.8)    X_train = X[:split_idx]    #
First 80%    X_test = X[split_idx:]    # Last 20%    y_train = y[:split_idx]
y_test = y[split_idx:]    print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")    # — Create and Train Model
model = RandomForestRegressor(n_estimators=100,
# Number of trees in the forest    max_depth=10,    # Maximum depth of
each tree    random_state=42,    # For reproducibility    n_jobs=-1
# Use all CPU cores    )    print("\nTraining... (this takes a few
seconds)")    model.fit(X_train, y_train)    # ← THIS IS LEARNING
print("Training complete!")    # — Make Predictions
y_pred_train = model.predict(X_train)    # On training
data    y_pred_test = model.predict(X_test)    # On NEW test data    # —
Calculate Errors —————    train_mae =
mean_absolute_error(y_train, y_pred_train)    test_mae =
mean_absolute_error(y_test, y_pred_test)    train_rmse =
np.sqrt(mean_squared_error(y_train, y_pred_train))    test_rmse =
np.sqrt(mean_squared_error(y_test, y_pred_test))    print(f"\nResults for
{home_name}:")    print(f"    Train MAE: {train_mae:.4f} kWh")    print(f"    Test
MAE: {test_mae:.4f} kWh ← This is what matters")    print(f"    Train RMSE:
{train_rmse:.4f} kWh")    print(f"    Test RMSE: {test_rmse:.4f} kWh")    #
— Check for Overfitting —————    # If train MAE << test MAE,
model is overfitting.    if train_mae < test_mae * 0.5:    print(" ⚠
WARNING: Possible overfitting detected")    else:    print(" ✓ No
significant overfitting")    # — Feature Importance —————
# Random Forest tells us which features matter most.    importance =
pd.DataFrame({    'feature':    FEATURE_COLUMNS,    'importance':
model.feature_importances_    }).sort_values('importance', ascending=False)
print(f"\nTop 5 important features for {home_name}:")    for _, row in
importance.head(5).iterrows():    bar = '█' * int(row['importance'] * 50)
print(f"    {row['feature']:20s} {bar} {row['importance']:.3f}")    # — Save
Model —————    model_path =
f'models/model_{home_name[-1].lower()}.pkl'    with open(model_path, 'wb') as
f:    pickle.dump(model, f)    print(f"\nModel saved: {model_path}")
# — Visualize Predictions —————    # Show actual vs predicted for
last 200 test points    fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(y_test[-200:], label='Actual', color='steelblue', lw=1.5)
ax.plot(y_pred_test[-200:], label='Predicted',    color='darkorange',
lw=1.5, linestyle='--')    ax.set_title(f'{home_name} — Actual vs Predicted
(last 200 points)', fontsize=13)    ax.set_xlabel('Time step')
ax.set_ylabel('Consumption (kWh)')    ax.legend()    ax.grid(True, alpha=0.3)
plt.tight_layout()    plt.savefig(f'forecast_{home_name.lower().replace(" ",
"")}.png',    dpi=150, bbox_inches='tight')    plt.close()
return {    'model':    model,    'test_mae':    test_mae,
'test_rmse':    test_rmse,    'train_mae':    train_mae,    'train_rmse':
train_rmse,    }# — Train models for all three homes —————results = {}for
name, path in [    ('Home A', 'homes/home_a.csv'),    ('Home B',
'homes/home_b.csv'),    ('Home C', 'homes/home_c.csv')]:    results[name] =
train_model(name, path)# — Comparison Table
print("\n" + "="*60)print("COMPARISON
TABLE")print("="*60)print(f"{'Home':<12} {'Train MAE':>12} {'Test MAE':>12}
{'Test RMSE':>12}")print("-"*60)for name, r in results.items():
print(f"{'name':<12} {r['train_mae']:>12.4f} {r['test_mae']:>12.4f}
{r['test_rmse']:>12.4f}")print("\n✅ Forecasting complete!")

```

How to run:

```
python 02_ml_forecasting.py
```

✓ Expected output: Three trained model files (*model_a.pkl*, *model_b.pkl*, *model_c.pkl*) in the *models/* folder. Three forecast charts. A comparison table of MAE/RMSE for each home.

Part 4: Detecting Unusual Energy Consumption

4.1 What is Anomaly Detection?

Anomaly detection is the task of finding data points that are very different from the rest. We don't need labeled examples — the model learns what "normal" looks like and flags anything that doesn't fit.

4.2 How Isolation Forest Works

Isolation Forest works by randomly splitting data. Normal points are hard to isolate (require many splits). Anomalous points are easy to isolate (few splits needed). Points that are isolated quickly get a high anomaly score.

```
Normal points:      Many splits needed → LOW anomaly score → Normal
Anomalous points:  Few splits needed → HIGH anomaly score → Anomaly
0.05 means we expect 5% of data to be anomalies. You set this based on your
domain knowledge.
```

4.3 Complete Anomaly Detection Code

```
# 03_anomaly_detection.py# —————# Uses
Isolation Forest to detect unusual energy# consumption patterns in each home.#
import pandas as pdimport numpy as npimport matplotlib.pyplot as pltfrom sklearn.ensemble import
IsolationForestfrom sklearn.metrics import classification_report,
confusion_matrixdef detect_anomalies(home_name, filepath):    """    Run
anomaly detection for one home.    """    print(f"\n{'='*50}")
print(f"Anomaly Detection: {home_name}")    print('='*50)    df =
pd.read_csv(filepath, parse_dates=['timestamp'])    # — Features for
anomaly detection    # We use the raw values, not lag features.    #
The model should detect unusual consumption directly.    anomaly_features = [
'consumption_kwh',    'rolling_mean_4',    'rolling_std_4',
'hour',    ]    X = df[anomaly_features].values    y_true =
df['is_anomaly'].values    # Ground truth labels    # — Train Isolation
Forest    # contamination: expected fraction of anomalies
# n_estimators: number of isolation trees    # random_state: for
reproducibility    model = IsolationForest(    contamination=0.02,    # We
expect ~2% anomalies    n_estimators=100,    random_state=42    )
print("Training Isolation Forest...")    model.fit(X)    # — Get
predictions    # Isolation Forest returns: -1 for
anomaly, 1 for normal    # We convert to: 1 for anomaly, 0 for normal
raw_predictions = model.predict(X)    y_pred = (raw_predictions ==
-1).astype(int)    # Anomaly scores (lower = more anomalous)
anomaly_scores = model.score_samples(X)    # Normalize to 0-1 range (1 = most
anomalous)    anomaly_scores_normalized = 1 - (    (anomaly_scores -
anomaly_scores.min()) /    (anomaly_scores.max() - anomaly_scores.min())
)    # — Evaluate    n_detected =
y_pred.sum()    n_actual = y_true.sum()    print(f"\nActual anomalies:
{n_actual}")    print(f"Detected anomalies: {n_detected}")    #
Classification report (precision, recall, F1)    print("\nClassification
Report:")    print(classification_report(y_true, y_pred,
target_names=['Normal', 'Anomaly']))    # Confusion matrix:    # [[True
Negative, False Positive],    # [False Negative, True Positive]]    cm =
confusion_matrix(y_true, y_pred)    print("Confusion Matrix:")    print(f"
True Negative: {cm[0,0]} (correctly identified normal)")    print(f" False
Positive: {cm[0,1]} (flagged normal as anomaly)")    print(f" False Negative:
{cm[1,0]} (missed actual anomaly)")    print(f" True Positive: {cm[1,1]}
(correctly detected anomaly)")    # — Visualize
```

```

df['predicted_anomaly'] = y_pred
df['anomaly_score'] = anomaly_scores_normalized
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 8)) # Plot 1: Consumption with anomaly
markers sample = df.head(500) normal_pts =
sample[sample['predicted_anomaly'] == 0] anomaly_pts =
sample[sample['predicted_anomaly'] == 1] ax1.plot(sample.index,
sample['consumption_kwh'], color='steelblue', lw=1,
label='Consumption', zorder=1) ax1.scatter(anomaly_pts.index,
anomaly_pts['consumption_kwh'], color='red', s=80, zorder=5,
label='Detected Anomaly') ax1.set_title(f'{home_name} - Detected Anomalies',
fontsize=13) ax1.set_ylabel('kWh') ax1.legend() ax1.grid(True,
alpha=0.3) # Plot 2: Anomaly scores ax2.plot(sample.index,
sample['anomaly_score'], color='darkorange', lw=1)
ax2.axhline(y=0.7, color='red', linestyle='--', label='Alert
threshold (0.7)') ax2.set_title(f'{home_name} - Anomaly Scores',
fontsize=13) ax2.set_ylabel('Anomaly Score') ax2.set_xlabel('Time step')
ax2.legend() ax2.grid(True, alpha=0.3) plt.tight_layout()
plt.savefig(f'anomaly_{home_name.lower().replace(" ", "_")}.png',
dpi=150, bbox_inches='tight') plt.close() return { 'model':
model, 'n_detected': n_detected, 'precision': cm[1,1] / (cm[0,1]
+ cm[1,1] + 1e-6), 'recall': cm[1,1] / (cm[1,0] + cm[1,1] + 1e-6),
}# — Run for all homes anomaly_results = {}for name,
path in [ ('Home A', 'homes/home_a.csv'), ('Home B', 'homes/home_b.csv'),
('Home C', 'homes/home_c.csv')]: anomaly_results[name] =
detect_anomalies(name, path)print("\n✅ Anomaly detection complete!")

```

Part 5: Non-IID Data and Why It Matters for Federated Learning

5.1 What is IID vs Non-IID Data?

IID stands for Independent and Identically Distributed. This is what most textbook ML examples assume — all data comes from the same distribution. In real federated learning scenarios, data is Non-IID: each client (home) has data from a different distribution.

IID (textbook assumption): All homes have similar consumption patterns Mean: ~2.0 kWh for everyone → Federated Learning is trivial
Non-IID (real world):
Home A: Mean = 1.5 kWh (small family) Home B: Mean = 3.5 kWh (large family)
Home C: Mean = 2.0 kWh (variable schedule) → Federated Learning must handle distribution mismatch!

 This is a key challenge of Federated Learning. If data were IID, you could just train centrally. Non-IID is what makes FL genuinely difficult and interesting.

5.2 Comparison Script — Local vs Centralized

```
# 04_virtual_homes.py# —————# Compares
three training approaches:# 1. Local:      Each home trains its own model# 2.
Centralized: All data pooled, one model# 3. Federated: (Placeholder — done
in Part 6)# —————import pandas as
pdimport numpy as npfrom sklearn.ensemble import RandomForestRegressorfrom
sklearn.metrics import mean_absolute_error, mean_squared_errorimport
matplotlib.pyplot as pltFEATURES = ['lag_1','lag_4','lag_96',
'rolling_mean_4','rolling_mean_96',
'rolling_std_4','hour','day_of_week','is_weekend','month']TARGET =
'consumption_kwh'def load_split(filepath):    """Load CSV and return train/test
splits."""    df = pd.read_csv(filepath)    df = df.dropna()    X =
df[FEATURES].values    y = df[TARGET].values    split = int(len(X) * 0.8)
return X[:split], X[split:], y[:split], y[split:]def evaluate(model, X_test,
y_test):    preds = model.predict(X_test)    return {        'mae':
mean_absolute_error(y_test, preds),        'rmse':
np.sqrt(mean_squared_error(y_test, preds)),    }def train_rf(X_train, y_train):
m = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42,
n_jobs=-1)    m.fit(X_train, y_train)    return m# — Load data
print("Loading data...")Xa_tr, Xa_te, ya_tr,
ya_te = load_split('homes/home_a.csv')Xb_tr, Xb_te, yb_tr, yb_te =
load_split('homes/home_b.csv')Xc_tr, Xc_te, yc_tr, yc_te =
load_split('homes/home_c.csv')# — Approach 1: Local models
print("\nApproach 1: Local models (each home trains
alone)")model_a = train_rf(Xa_tr, ya_tr)model_b = train_rf(Xb_tr, yb_tr)model_c
= train_rf(Xc_tr, yc_tr)local_results = {    'Home A (local)':
evaluate(model_a, Xa_te, ya_te),    'Home B (local)': evaluate(model_b, Xb_te,
yb_te),    'Home C (local)': evaluate(model_c, Xc_te, yc_te),}# — Approach 2:
Centralized
print("Approach 2: Centralized (all data
combined)")X_all_tr = np.vstack([Xa_tr, Xb_tr, Xc_tr])y_all_tr =
np.concatenate([ya_tr, yb_tr, yc_tr])central_model = train_rf(X_all_tr,
y_all_tr)centralized_results = {    'Home A (central)': evaluate(central_model,
Xa_te, ya_te),    'Home B (central)': evaluate(central_model, Xb_te, yb_te),
'Home C (central)': evaluate(central_model, Xc_te, yc_te),}# — Print
comparison
print("\n" + "="*65)print("COMPARISON:
LOCAL vs CENTRALIZED")print("="*65)print(f"{'Setup':<25} {'MAE':>10}
{'RMSE':>10}")print("-"*65)for label, r in (**local_results,
**centralized_results).items():    print(f"{label:<25} {r['mae']:>10.4f}
{r['rmse']:>10.4f}")# — Visualize
homes =
['Home A', 'Home B', 'Home C']local_maes    = [local_results[f'{h}']
```

```

(local)']['mae'] for h in homes]central_maes = [centralized_results[f'{h}
(central)']['mae'] for h in homes]x = np.arange(len(homes))width = 0.35fig, ax
= plt.subplots(figsize=(10, 6))bars1 = ax.bar(x - width/2, local_maes,
width, label='Local', color='steelblue')bars2 = ax.bar(x + width/2,
central_maes, width, label='Centralized',
color='darkorange')ax.set_xlabel('Home')ax.set_ylabel('MAE
(kWh)')ax.set_title('Local vs Centralized: MAE
Comparison')ax.set_xticks(x)ax.set_xticklabels(homes)ax.legend()ax.grid(True,
alpha=0.3, axis='y')# Add value labels on barsfor bar in bars1:
ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=10)for bar in
bars2: ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
f'{bar.get_height():.3f}', ha='center', va='bottom',
fontsize=10)plt.tight_layout()plt.savefig('local_vs_centralized.png', dpi=150,
bbox_inches='tight')plt.show()print("\n✅ Comparison complete!")

```

Part 6: Implementing Federated Learning

6.1 Understanding FedAvg — The Core Algorithm

FedAvg (Federated Averaging) is the most common Federated Learning algorithm. It works like this:

```
Round 1: Server: "Everyone start with these initial weights" Client A: trains locally → gets weights_A Client B: trains locally → gets weights_B Client C: trains locally → gets weights_C Server: global_weights = (weights_A + weights_B + weights_C) / 3
Round 2: Server: "Everyone start with Round 1 global weights" Client A: trains locally from global_weights → updates weights_A ...repeat...
```

6.2 Manual FedAvg (Without Flower)

Before using the Flower library, we implement FedAvg manually to deeply understand what happens:

```
# 05_federated_learning.py (Part 1: Manual FedAvg)#
# Manual implementation of Federated Learning# WITHOUT using Flower library.# This teaches you exactly what FL does step by step.#
import pandas as pdimport numpy as npimport matplotlib.pyplot as pltfrom sklearn.ensemble import RandomForestRegressorfrom sklearn.metrics import mean_absolute_error, mean_squared_errorFEATURES = ['lag_1', 'lag_4', 'lag_96', 'rolling_mean_4', 'rolling_mean_96', 'rolling_std_4', 'hour', 'day_of_week', 'is_weekend', 'month']TARGET = 'consumption_kwh'# CRITICAL CONCEPT: Model Weights# For traditional ML (not deep learning), we can't# average the weights directly like in neural networks.# Instead, we use a practical approach:## Strategy 1 (Simple): Train on combined pseudo-data# - Each home sends a SAMPLE of its predictions# - Server trains on combined samples# - This simulates the spirit of federated averaging## Strategy 2 (Proper for sklearn): Vote/ensemble# - Keep all local models# - Global prediction = weighted average of all modelsprint("Federated Learning - Manual FedAvg Implementation")print("="*55)# Load data
def load_data(path):
    df = pd.read_csv(path).dropna()
    X = df[FEATURES].values
    y = df[TARGET].values
    split = int(len(X) * 0.8)
    return X[:split], X[split:], y[:split], y[split:]
Xa_tr, Xa_te, ya_tr, ya_te = load_data('homes/home_a.csv')
Xb_tr, Xb_te, yb_tr, yb_te = load_data('homes/home_b.csv')
Xc_tr, Xc_te, yc_tr, yc_te = load_data('homes/home_c.csv')
# Step 1: Initialize global "knowledge"
# In the first round, we create a base model# from a small combined sample.print("\nStep 1: Initializing global model...")# Take a small random sample from each home# (simulating only summary info being shared)sample_size = 200Xa_sample = Xa_tr[:sample_size]ya_sample = ya_tr[:sample_size]Xb_sample = Xb_tr[:sample_size]yb_sample = yb_tr[:sample_size]Xc_sample = Xc_tr[:sample_size]yc_sample = yc_tr[:sample_size]
# Step 2: Federated Rounds
class FederatedClient:
    """Represents one FL client (one home)."""
    def __init__(self, name, X_train, y_train, X_test, y_test):
        self.name = name
        self.X_train = X_train
        self.y_train = y_train
        self.X_test = X_test
        self.y_test = y_test
        self.model = None
        self.history = [] # Track MAE over rounds
    def local_train(self, global_params=None, n_trees=50):
        """Train locally.
        global_params: If provided, we use global model as starting point.
        # In sklearn, we can't truly "resume" training from global weights.
        # We simulate this by:
        # 1. If global_params provided: use global model predictions as "pseudo-labels" and train on augmented dataset.
        # 2. This mimics the warming effect of FL.
        if global_params is not None and 'model' in global_params:
            # Use global model to generate pseudo-predictions
            global_preds = global_params['model'].predict(self.X_train)
            # Mix local labels with global predictions (70% local, 30% global)
            y_mixed = 0.7 * self.y_train + 0.3 * global_preds
            self.model = RandomForestRegressor().fit(self.X_train, y_mixed)
        else:
            self.model = RandomForestRegressor().fit(self.X_train, self.y_train)
        self.history.append(mean_absolute_error(self.model.predict(self.X_test), self.y_test))
```

```

self.y_train + 0.3 * global_preds          else:                y_mixed =
self.y_train                             self.model = RandomForestRegressor(
n_estimators=n_trees,                    max_depth=8,            random_state=42,
n_jobs=-1)                             self.model.fit(self.X_train, y_mixed)
# Evaluate locally                      preds = self.model.predict(self.X_test)          mae =
mean_absolute_error(self.y_test, preds)    self.history.append(mae)
print(f" {self.name}: Local MAE = {mae:.4f}")    return self.model
def get_predictions_sample(self, X_sample):    """Return model predictions
on a shared sample."""    return self.model.predict(X_sample)class
FederatedServer:    """The central server that aggregates client updates."""
def __init__(self, clients):                self.clients        = clients
self.global_model = None                    self.round_maes      = []    # Track global
performance                                # Create a shared evaluation set (from first client, not
training data)    self.X_eval = clients[0].X_test[:100]    self.y_eval
= clients[0].y_test[:100]    def aggregate(self, shared_X, shared_y):
"""    FedAvg: Train global model on averaged client outputs.
In practice with sklearn:                - Collect predictions from all clients on a
shared dataset                - Average those predictions                - Train a new global
model on (features, averaged predictions)    """    # Get predictions
from each client    all_predictions = np.array([
client.get_predictions_sample(shared_X)    for client in self.clients
])    # Average across clients (this IS FedAvg for predictions)
averaged_predictions = all_predictions.mean(axis=0)    # Train
global model on averaged predictions    self.global_model =
RandomForestRegressor(n_estimators=100,    max_depth=10,
random_state=42,    n_jobs=-1)
self.global_model.fit(shared_X, averaged_predictions)    # Evaluate
global model    global_preds = self.global_model.predict(self.X_eval)
global_mae = mean_absolute_error(self.y_eval, global_preds)
self.round_maes.append(global_mae)    print(f" → Global MAE:
{global_mae:.4f}")    return self.global_model# — Create clients
clients = [    FederatedClient('Home A', Xa_tr,
ya_tr, Xa_te, ya_te),    FederatedClient('Home B', Xb_tr, yb_tr, Xb_te, yb_te),
FederatedClient('Home C', Xc_tr, yc_tr, Xc_te, yc_te),]server =
FederatedServer(clients)# Shared evaluation dataset (public, doesn't contain
private data)# In real FL, this is a small public dataset used for
aggregationX_shared = np.vstack([Xa_sample, Xb_sample, Xc_sample])y_shared =
np.concatenate([ya_sample, yb_sample, yc_sample])# — Run Federated Learning
Rounds    N_ROUNDS = 5global_params = Noneprint("\n" +
"*55)print("FEDERATED LEARNING - Running Rounds")print("*55)for round_num
in range(1, N_ROUNDS + 1):    print(f"\n--- Round {round_num}/{N_ROUNDS} ---")
# Step 1: Distribute global model to all clients    print(" Broadcasting
global model...")    # Step 2: Each client trains locally    print(" Local
training:")    for client in clients:
client.local_train(global_params=global_params, n_trees=50)    # Step 3:
Server aggregates    print(" Aggregating:")    global_model =
server.aggregate(X_shared, y_shared)    global_params = {'model':
global_model}# — Final evaluation    print("\n" +
"*55)print("FINAL RESULTS")print("*55)def final_eval(model, X_test,
y_test):    preds = model.predict(X_test)    mae = mean_absolute_error(y_test,
preds)    rmse = np.sqrt(mean_squared_error(y_test, preds))    return mae,
rmseprint(f"\n{'Approach':<25} {'MAE':>10} {'RMSE':>10}")print("-"*50)for
client in clients:    mae, rmse = final_eval(client.model, client.X_test,
client.y_test)    print(f"{client.name + ' (local)':<25} {mae:>10.4f}
{rmse:>10.4f}")# Global model evaluation on each homefor client in clients:
mae, rmse = final_eval(global_model, client.X_test, client.y_test)
print(f"{client.name + ' (federated)':<25} {mae:>10.4f} {rmse:>10.4f}")# —
Visualize convergence    fig, (ax1, ax2) = plt.subplots(1,
2, figsize=(14, 5))rounds = list(range(1, N_ROUNDS + 1))for client in clients:
ax1.plot(rounds, client.history, marker='o',
label=client.name)ax1.set_title('Local MAE per Round')ax1.set_xlabel('FL
Round')ax1.set_ylabel('MAE (kWh)')ax1.legend()ax1.grid(True,
alpha=0.3)ax2.plot(rounds, server.round_maes, marker='s',
color='darkgreen', linewidth=2, label='Global Model')ax2.set_title('Global
Model MAE Convergence')ax2.set_xlabel('FL Round')ax2.set_ylabel('MAE
(kWh)')ax2.legend()ax2.grid(True,
alpha=0.3)plt.tight_layout()plt.savefig('federated_learning.png', dpi=150,
bbox_inches='tight')plt.show()print("\n✅ Federated Learning complete!")

```


Part 7: Understanding Model Predictions with SHAP

7.1 Why Do We Need Explainability?

A model that says "4.8 kWh" without explanation is not useful for decision-making. Engineers and operators need to know WHY the model made that prediction. SHAP provides this explanation.

7.2 How SHAP Works

SHAP calculates the marginal contribution of each feature to the final prediction. Positive SHAP values push the prediction higher. Negative SHAP values push it lower. The sum of all SHAP values equals the prediction minus the base value.

```
Example: Base prediction (average):      1.8 kWh      SHAP contributions:  +
lag_1 (recent consumption):    +0.81  + rolling_mean_4 (avg):      +0.42  +
hour (8 AM peak):              +0.21  - day_of_week (Wed, moderate): -0.08
Final prediction: 1.8 + 0.81 + 0.42 + 0.21 - 0.08 = 3.16 kWh
```

7.3 Complete SHAP Code

```
# 06_shap_explainability.py# —————#
Explains model predictions using SHAP.# Makes the "black box" transparent.#
import pandas as pdimport numpy as npimport matplotlib.pyplot as pltimport shapimport pickleFEATURES = [
'lag_1','lag_4','lag_96',      'rolling_mean_4','rolling_mean_96',
'rolling_std_4','hour','day_of_week','is_weekend','month']TARGET =
'consumption_kwh'def explain_model(home_name, model_path, data_path):
    """Generate SHAP explanations for one home's model."""
    print(f"\n{'='*50}")      print(f"SHAP Explanation: {home_name}")
    print('='*50)      # Load model and data      with open(model_path, 'rb') as f:
    model = pickle.load(f)      df = pd.read_csv(data_path).dropna()      X =
    pd.DataFrame(df[FEATURES].values, columns=FEATURES)      y = df[TARGET].values
    # — Create SHAP Explainer —————      # TreeExplainer is optimized
    for tree-based models      # (Random Forest, XGBoost, etc.)      print("Creating
    SHAP explainer...")      explainer = shap.TreeExplainer(model)      # —
    Calculate SHAP values —————      # Use a sample for speed — full
    dataset takes too long      sample_size = 500      X_sample = X.iloc[:sample_size]
    print(f"Calculating SHAP values for {sample_size} samples...")      shap_values =
    explainer.shap_values(X_sample)      print(f"SHAP values shape:
    {shap_values.shape}")      # Shape: (n_samples, n_features) — one SHAP value per
    feature per sample      # — Global Feature Importance —————      #
    Mean absolute SHAP value per feature      mean_shap =
    np.abs(shap_values).mean(axis=0)      importance_df = pd.DataFrame({
    'feature':      FEATURES,      'importance': mean_shap,
    }).sort_values('importance', ascending=False)      print(f"\nGlobal Feature
    Importance (SHAP):")      for _, row in importance_df.iterrows():      bar =
    '█' * int(row['importance'] / mean_shap.max() * 30)      print(f"
    {row['feature']:25s} {bar:30s} {row['importance']:.4f}")      # — Explain
    ONE specific prediction —————      # Pick a sample to explain in detail
    sample_idx = 100 # 101st data point      sample_X =
    X_sample.iloc[[sample_idx]]      prediction = model.predict(sample_X)[0]
    base_value = explainer.expected_value      print(f"\nDetailed explanation for
    sample {sample_idx}:")      print(f"      Base value (average prediction):
    {base_value:.4f} kWh")      print(f"      Actual prediction:
    {prediction:.4f} kWh")      print(f"      Difference from base:
    {prediction - base_value:+.4f} kWh")      print(f"\n      Feature contributions:")
    sample_shap = shap_values[sample_idx]      feature_contributions =
    list(zip(FEATURES, sample_shap, sample_X.values[0]))
    feature_contributions.sort(key=lambda x: abs(x[1]), reverse=True)      for
```

```

feat, shap_val, feat_val in feature_contributions:
    direction = '+' if
shap_val > 0 else '-'
    bar = '█' * int(abs(shap_val) / max(abs(s) for
_, s, _ in feature_contributions) * 20)
    print(f"    {feat:25s}
value={feat_val:6.2f} SHAP={shap_val:+.4f} {bar}")
    # — Visualize
fig, axes = plt.subplots(1, 2, figsize=(16,
7))
    # Plot 1: Summary bar plot (global importance)
    ax = axes[0]
    colors = ['#2E75B6' if v > 0 else '#C0392B'
    for v in
importance_df['importance']]
    ax.barh(importance_df['feature'],
importance_df['importance'],
color='steelblue')
    ax.set_xlabel('Mean |SHAP value|')
    ax.set_title(f'{home_name} – Feature
Importance (SHAP)')
    ax.invert_yaxis()
    ax.grid(True, alpha=0.3, axis='x')
    # Plot 2: Waterfall for one prediction
    ax = axes[1]
    sorted_feats =
[fc[0] for fc in feature_contributions]
    sorted_shaps = [fc[1] for fc in
feature_contributions]
    bar_colors = ['forestgreen' if s > 0 else 'crimson'
for s in sorted_shaps]
    ax.barh(sorted_feats, sorted_shaps,
color=bar_colors)
    ax.axvline(x=0, color='black', linewidth=0.8)
    ax.set_xlabel('SHAP value (impact on prediction)')
    ax.set_title(f'{home_name} – Single Prediction Explanation')
    ax.invert_yaxis()
    ax.grid(True, alpha=0.3, axis='x')
    plt.tight_layout()
    plt.savefig(f'shap_{home_name.lower().replace(" ",
"_")}.png',
dpi=150, bbox_inches='tight')
    plt.close()
    return {
        'top_feature': importance_df.iloc[0]['feature'],
        'prediction': prediction,
        'base_value': base_value,
        'shap_values': shap_values,
        'feature_contributions':
feature_contributions,
    }
    # — Run SHAP for all homes
for
name, model_p, data_p in [
    ('Home A', 'models/model_a.pkl',
'homes/home_a.csv'),
    ('Home B', 'models/model_b.pkl', 'homes/home_b.csv'),
    ('Home C', 'models/model_c.pkl', 'homes/home_c.csv')
]:
    explain_model(name,
model_p, data_p)
print("\n✅ SHAP explanation complete!")

```

Part 8: Building the Energy Agent

8.1 What Makes Something an "Agent"?

An AI agent has three properties: it perceives its environment (reads sensor data, predictions, anomaly scores), it reasons (applies logic or ML to make a decision), and it acts (outputs a recommendation, triggers an alert, sends a command).

8.2 Complete Agent Code

```
# 07_energy_agent.py# -----# Rule-based
Energy Agent that:# 1. Takes forecasts and anomaly scores as input# 2. Reasons
about the situation# 3. Outputs recommendations#
-----import pickleimport pandas as
pdimport numpy as npimport shapFEATURES = [ 'lag_1','lag_4','lag_96',
'rolling_mean_4','rolling_mean_96',
'rolling_std_4','hour','day_of_week','is_weekend','month']class EnergyAgent:
"""    A rule-based agent for smart home energy management.    This is
intentionally simple - in GridGuard, this would    be replaced by a more
sophisticated LLM-based agent.    But the ARCHITECTURE (observe -> reason -> act)
is the same.    """    def __init__(self, home_id, model_path,
anomaly_model_path=None):        self.home_id = home_id        # Load
forecasting model        with open(model_path, 'rb') as f:
self.forecast_model = pickle.load(f)        # Create SHAP explainer
self.explainer = shap.TreeExplainer(self.forecast_model)        #
Thresholds for decision making        self.ANOMALY_THRESHOLD_LOW    = 0.5
self.ANOMALY_THRESHOLD_HIGH    = 0.75        self.FORECAST_HIGH_THRESHOLD    = 3.5
# kWh        self.FORECAST_VERY_HIGH    = 5.0    # kWh
print(f"Energy Agent initialized for {home_id}")    def observe(self,
features_dict):        """        OBSERVE: Gather current state.        Takes a
dictionary of feature values and returns        a structured observation.
"""        features_df = pd.DataFrame([features_dict])        X =
features_df[FEATURES].values        # Get prediction        prediction =
self.forecast_model.predict(X)[0]        # Get SHAP explanation
shap_values = self.explainer.shap_values(X)[0]        feature_impacts = sorted(
zip(FEATURES, shap_values),        key=lambda x: abs(x[1]),
reverse=True        )        observation = {        'home_id':
self.home_id,        'predicted_kwh':    round(prediction, 3),
'top_features':    feature_impacts[:3],        'raw_features':
features_dict,        }        return observation    def reason(self,
observation, anomaly_score):        """        REASON: Analyze the observation
and determine severity.        Returns a structured analysis with:
- severity level        - detected conditions        - reasoning explanation
"""        pred = observation['predicted_kwh']        conditions = []
reasoning = []        # ----- Check anomaly -----
if anomaly_score > self.ANOMALY_THRESHOLD_HIGH:
conditions.append('HIGH_ANOMALY')        reasoning.append(f"Anomaly score
{anomaly_score:.2f} exceeds high threshold.")        elif anomaly_score >
self.ANOMALY_THRESHOLD_LOW:
conditions.append('MODERATE_ANOMALY')
reasoning.append(f"Anomaly score {anomaly_score:.2f} is moderately elevated.")
# ----- Check forecast -----        if pred >
self.FORECAST_VERY_HIGH:
conditions.append('VERY_HIGH_FORECAST')
reasoning.append(f"Predicted {pred:.2f} kWh far exceeds normal range.")
elif pred > self.FORECAST_HIGH_THRESHOLD:
conditions.append('HIGH_FORECAST')        reasoning.append(f"Predicted
{pred:.2f} kWh is above normal range.")        # ----- Check feature
explanations -----        for feat, shap_val in
observation['top_features']:        if feat == 'lag_1' and shap_val > 0.5:
conditions.append('RECENT_SURGE')        reasoning.append("Recent
consumption spike detected.")        break        # -----
```

```

Determine severity ----- if 'HIGH_ANOMALY' in conditions or
'VERY_HIGH_FORECAST' in conditions: severity = 'CRITICAL'
elif 'MODERATE_ANOMALY' in conditions or 'HIGH_FORECAST' in conditions:
severity = 'WARNING' elif 'RECENT_SURGE' in conditions:
severity = 'ADVISORY' else: severity = 'NORMAL'
return { 'severity': severity, 'conditions':
conditions, 'reasoning': reasoning, } def act(self,
observation, analysis): """ ACT: Generate a concrete
recommendation. Maps severity + conditions → actionable
recommendation. severity = analysis['severity']
conditions = analysis['conditions'] pred =
observation['predicted_kwh'] # — Generate recommendation
----- if severity == 'CRITICAL': if 'HIGH_ANOMALY' in
conditions and 'VERY_HIGH_FORECAST' in conditions: action =
"IMMEDIATE INSPECTION REQUIRED" message = (
f"Critical energy anomaly detected at {self.home_id}. "
f"Predicted consumption ({pred:.2f} kWh) is extremely high. "
"Check for faulty appliances, meter tampering, or unusual activity. "
"Consider temporarily reducing load." ) elif
'HIGH_ANOMALY' in conditions: action = "INVESTIGATE IMMEDIATELY"
message = ( f"High anomaly score at {self.home_id}. "
"Current consumption pattern is highly unusual. " "Check all
high-draw appliances." ) else: action
= "REDUCE LOAD" message = ( f"Very high
consumption predicted ({pred:.2f} kWh) at {self.home_id}. "
"Consider deferring high-energy tasks." ) elif
severity == 'WARNING': action = "MONITOR CLOSELY" message
= ( f"Elevated energy usage at {self.home_id} "
f"(predicted: {pred:.2f} kWh). " "Review active appliances and
consider load scheduling." ) elif severity ==
'ADVISORY': action = "ADVISORY" message = (
f"Recent consumption surge detected at {self.home_id}. "
"Monitor for continued high usage." ) else:
action = "NORMAL — NO ACTION REQUIRED" message = (
f"{self.home_id} operating normally. " f"Predicted consumption:
{pred:.2f} kWh." ) return { 'home_id':
self.home_id, 'severity': severity, 'action':
action, 'message': message, 'prediction': pred,
} def run(self, features_dict, anomaly_score): """ Full
agent cycle: Observe → Reason → Act. Returns the complete
recommendation. """ observation = self.observe(features_dict)
analysis = self.reason(observation, anomaly_score) decision =
self.act(observation, analysis) return decision# — Demo: Run the agent
-----if __name__ == '__main__': # Load one home's
data to get a real feature sample df =
pd.read_csv('homes/home_a.csv').dropna() print("="*60) print("ENERGY
AGENT DEMONSTRATION") print("="*60) agent_a = EnergyAgent('Home A',
'models/model_a.pkl') # Test 1: Normal situation print("\n--- Test 1:
Normal Consumption ---") normal_sample = df[FEATURES].iloc[500].to_dict()
result = agent_a.run(normal_sample, anomaly_score=0.2) print(f" Severity:
{result['severity']}") print(f" Action: {result['action']}")
print(f" Message: {result['message']}") # Test 2: High anomaly
print("\n--- Test 2: High Anomaly Detected ---") anomaly_sample =
df[FEATURES].iloc[500].to_dict() # Simulate high recent consumption
anomaly_sample['lag_1'] = 9.5 anomaly_sample['rolling_mean_4'] = 7.2
result = agent_a.run(anomaly_sample, anomaly_score=0.85) print(f" Severity:
{result['severity']}") print(f" Action: {result['action']}")
print(f" Message: {result['message']}") print("\n✅ Agent
demonstration complete!")

```

Part 9: Exposing Your AI as a Web Service with FastAPI

9.1 Why an API?

Right now your ML code only runs when you execute a Python script. An API turns it into a running service that any application (a dashboard, a mobile app, another microservice) can call over HTTP in real time.

```
Without API: python 02_ml_forecasting.py → output printed to terminal →
can't be used by other apps
With FastAPI: python 08_api.py → server runs at
http://localhost:8000 → browser/dashboard calls: GET /forecast/home-a → gets
JSON response instantly
```

9.2 How FastAPI Works

FastAPI uses Python decorators (`@app.get`) to map URL paths to Python functions. When someone calls the URL, FastAPI runs the function and returns the result as JSON.

9.3 Complete FastAPI Code

```
# 08_api.py# —————# FastAPI server
exposing all AI capabilities# as REST API endpoints.#
from fastapi import FastAPI,
HTTPExceptionfrom fastapi.middleware.cors import CORSMiddlewareimport pandas as
pdimport numpy as npimport pickleimport shapfrom sklearn.ensemble import
IsolationForest# — Create FastAPI app —————app = FastAPI(
title="EnergyGuard Mini API", description="Privacy-Preserving Smart Home
Energy Intelligence", version="1.0.0")# — Allow cross-origin requests
# This lets Streamlit (on a different port) call our
APIapp.add_middleware( CORSMiddleware, allow_origins=["*"],
allow_methods=["*"], allow_headers=["*"],)# — Configuration
FEATURES = [ 'lag_1','lag_4','lag_96',
'rolling_mean_4','rolling_mean_96',
'rolling_std_4','hour','day_of_week','is_weekend','month']HOME_DATA = {
'home-a': 'homes/home_a.csv', 'home-b': 'homes/home_b.csv', 'home-c':
'homes/home_c.csv',}HOME_MODELS = { 'home-a': 'models/model_a.pkl',
'home-b': 'models/model_b.pkl', 'home-c': 'models/model_c.pkl',)# — Load
models at startup —————# We load once to avoid reloading on
every requestprint("Loading models...")models = {}explainers = {}for
home_id, path in HOME_MODELS.items(): try: with open(path, 'rb') as
f: models[home_id] = pickle.load(f) explainers[home_id] =
shap.TreeExplainer(models[home_id]) print(f" Loaded: {home_id}")
except FileNotFoundError: print(f" WARNING: Model not found for
{home_id}")# — Helper functions —————def
get_latest_features(home_id: str) -> pd.DataFrame: """Load the most recent
data point for a home.""" path = HOME_DATA.get(home_id) if not path:
raise HTTPException(status_code=404, detail=f"Home
{home_id} not found") df = pd.read_csv(path).dropna() return
df[FEATURES].iloc[-1] # Most recent rowdef validate_home(home_id: str):
"""Raise error if home_id is invalid.""" if home_id not in HOME_DATA:
raise HTTPException( status_code=404, detail=f"Home
{home_id}' not found. " f"Valid IDs:
{list(HOME_DATA.keys())}") if home_id not in models: raise
HTTPException( status_code=503, detail=f"Model for
{home_id}' not loaded" )#
# API ENDPOINTS#
@app.get("/")def root():
"""Welcome endpoint.""" return { "service": "EnergyGuard Mini",
"version": "1.0.0", "endpoints": [ "/homes",
```

```

"/forecast/{home_id}",                "/anomalies/{home_id}",
"/explanation/{home_id}",                "/recommendation/{home_id}",
"/comparison",                        "/federated/status",
]
@app.get("/")def get_homes():            """List all available homes with basic
stats."""    homes_info = []    for home_id, path in HOME_DATA.items():
try:        df = pd.read_csv(path).dropna()        homes_info.append({
"home_id":            home_id,            "data_points":            len(df),
"avg_consumption":    round(df['consumption_kwh'].mean(), 3),
"max_consumption":    round(df['consumption_kwh'].max(), 3),
"anomaly_count":        int(df['is_anomaly'].sum()),            "model_loaded":
home_id in models,            })    except Exception as e:
homes_info.append({"home_id": home_id, "error": str(e)})    return {"homes":
homes_info}@app.get("/forecast/{home_id}")def get_forecast(home_id: str):
"""    Predict next energy consumption for a home.    Returns the predicted
kWh value.    """    validate_home(home_id)    features =
get_latest_features(home_id)    X = features.values.reshape(1, -1)
prediction = float(models[home_id].predict(X)[0])    return {
"home_id":            home_id,            "predicted_consumption_kwh": round(prediction,
3),            "unit": "kWh",            "interval": "15 minutes",
}@app.get("/anomalies/{home_id}")def get_anomalies(home_id: str, n_recent: int
= 96):    """    Get anomaly detection results for recent data.
n_recent: how many recent data points to check (default: 96 = 24h)    """
validate_home(home_id)    path = HOME_DATA[home_id]    df =
pd.read_csv(path).dropna()    recent = df.tail(n_recent)    # Train
Isolation Forest on recent data    X = recent[FEATURES].values    iso =
IsolationForest(contamination=0.02, random_state=42, n_estimators=100)
iso.fit(X)    predictions = iso.predict(X)    anomaly_scores = 1 - (
(iso.score_samples(X) - iso.score_samples(X).min()) /
(iso.score_samples(X).max() - iso.score_samples(X).min() + 1e-6)    )
n_anomalies = int((predictions == -1).sum())    current_score =
float(anomaly_scores[-1])    current_status = "ANOMALY" if predictions[-1] ==
-1 else "NORMAL"    return {            "home_id":            home_id,
"period_checked": f>Last {n_recent} readings",            "n_anomalies":
n_anomalies,            "current_status": current_status,
"current_anomaly_score": round(current_score, 3),
}@app.get("/explanation/{home_id}")def get_explanation(home_id: str):    """
Get SHAP explanation for the latest prediction.    """
validate_home(home_id)    features = get_latest_features(home_id)    X =
pd.DataFrame([features], columns=FEATURES)    prediction =
float(models[home_id].predict(X.values)[0])    shap_values =
explainers[home_id].shap_values(X.values)[0]    feature_contributions =
sorted([{"feature": f, "shap_value": round(float(s), 4),
"direction": "increases" if s > 0 else "decreases"}    for f, s in
zip(FEATURES, shap_values)],            key=lambda x: abs(x["shap_value"]),
reverse=True    )    return {            "home_id":            home_id,
"prediction": round(prediction, 3),            "base_value":
round(float(explainers[home_id].expected_value), 3),            "top_factors":
feature_contributions[:5],            "explanation": f"The model predicted
{prediction:.2f} kWh. "            f"The main driver was
{feature_contributions[0]['feature']}' "            f"which
{feature_contributions[0]['direction']} the prediction.",
}@app.get("/recommendation/{home_id}")def get_recommendation(home_id: str):
"""    Full agent pipeline: forecast + anomaly + SHAP → recommendation.    """
validate_home(home_id)    path = HOME_DATA[home_id]    df =
pd.read_csv(path).dropna()    features = df[FEATURES].iloc[-1]    X =
features.values.reshape(1, -1)    prediction =
float(models[home_id].predict(X)[0])    X_recent =
df[FEATURES].tail(200).values    iso = IsolationForest(contamination=0.02,
random_state=42)    iso.fit(X_recent)    raw_scores = iso.score_samples(X)
anomaly_score = float(1 - (raw_scores - raw_scores.min()) /
(raw_scores.max() - raw_scores.min() + 1e-6))    is_anomaly = iso.predict(X)[0]
== -1    # Simple rule-based recommendation    if is_anomaly and prediction
> 5.0:        severity = "CRITICAL"        message = (f"Critical anomaly at
{home_id}. "        f"Predicted {prediction:.2f} kWh. "
"Immediate inspection recommended.")    elif is_anomaly:        severity =
"WARNING"        message = (f"Anomaly detected at {home_id}. "
f"Predicted {prediction:.2f} kWh. "        "Review high-consumption
appliances.")    elif prediction > 4.0:        severity = "ADVISORY"

```

```

message = (f"High predicted consumption ({prediction:.2f} kWh) at {home_id}. "
"Consider load shifting.") else: severity = "NORMAL" message
= f"{home_id} operating normally. Predicted: {prediction:.2f} kWh."
return { "home_id": home_id, "severity": severity,
"message": message, "prediction": round(prediction, 3),
"is_anomaly": is_anomaly, "anomaly_score": round(anomaly_score, 3),
}@app.get("/comparison")def get_comparison(): """Compare performance across
all homes.""" results = [] for home_id, path in HOME_DATA.items():
try: df = pd.read_csv(path).dropna() split = int(len(df)
* 0.8) X_test = df[FEATURES].iloc[split:].values y_test =
df['consumption_kwh'].iloc[split:].values preds =
models[home_id].predict(X_test) mae = float(np.mean(np.abs(preds
- y_test))) results.append({ "home_id": home_id,
"test_mae": round(mae, 4), }) except Exception as e:
results.append({"home_id": home_id, "error": str(e)}) return {"comparison":
results}@app.get("/federated/status")def get_federated_status(): """Return
simulated FL training status.""" return { "status": "COMPLETE",
"rounds": 5, "clients": ["home-a", "home-b", "home-c"],
"algorithm": "FedAvg", "note": "Privacy preserved: raw data never
left individual homes", }# — Run the server —————if
__name__ == "__main__": import uvicorn print("Starting EnergyGuard Mini
API...") print("Documentation: http://localhost:8000/docs")
uvicorn.run(app, host="0.0.0.0", port=8000, reload=False)

```

How to run:

```

python 08_api.py# Then open in browser:# http://localhost:8000/docs ←
Interactive API documentation# http://localhost:8000/homes ← List all
homes# http://localhost:8000/forecast/home-a

```

Part 10: Building the Interactive Dashboard

10.1 What is Streamlit?

Streamlit is a Python library that turns scripts into interactive web apps with no frontend experience required. Every time a user interacts (clicks a button, selects from a dropdown), Streamlit re-runs your Python script from top to bottom.

10.2 Complete Streamlit Dashboard

```
# 09_dashboard.py# —————# Interactive
Streamlit dashboard for EnergyGuard Mini.# Shows all AI capabilities in a
single web interface.# —————# Run:
streamlit run 09_dashboard.py#

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pickle
import shap
from sklearn.ensemble import IsolationForest, RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import warnings
warnings.filterwarnings('ignore')# — Page configuration
st.set_page_config(
    page_icon="🏠",
    layout="wide",
    initial_sidebar_state="expanded",
    # Use full browser width
)# — CSS styling
st.markdown("""<style>
    .metric-card {
    background: linear-gradient(135deg, #1F3864, #2E75B6);
    color: white;
    padding: 20px;
    border-radius: 12px;
    text-align: center;
    }
    .alert-critical {
    background-color: #FDEDEC;
    border-left: 6px
    solid #C0392B;
    padding: 12px;
    border-radius: 4px;
    }
    .alert-warning {
    background-color: #FDEBD0;
    border-left: 6px
    solid #CA6F1E;
    padding: 12px;
    border-radius: 4px;
    }
    .alert-normal {
    background-color: #D5F5E3;
    border-left: 6px
    solid #1E8449;
    padding: 12px;
    border-radius: 4px;
    }</style>""",
    unsafe_allow_html=True)# — Header
st.title("🏠 EnergyGuard
Mini")
st.caption("Privacy-Preserving Smart Home Energy Intelligence | GridGuard
Learning Project")
st.divider()# — Feature columns
FEATURES = [
    'lag_1', 'lag_4', 'lag_96',
    'rolling_mean_4', 'rolling_mean_96',
    'rolling_std_4', 'hour', 'day_of_week', 'is_weekend', 'month']
TARGET = 'consumption_kwh'
HOME_DATA = {
    'Home A': 'homes/home_a.csv',
    'Home B': 'homes/home_b.csv',
    'Home C': 'homes/home_c.csv',
}
HOME_MODELS = {
    'Home A': 'models/model_a.pkl',
    'Home B': 'models/model_b.pkl',
    'Home C': 'models/model_c.pkl',
}# — Cache data loading
@st.cache_data
def load_data():
    dfs = {}
    for name, path in HOME_DATA.items():
        try:
            dfs[name] = pd.read_csv(path).dropna()
        except FileNotFoundError:
            dfs[name] = None
    return dfs
@st.cache_resource
def load_models():
    mdls = {}
    expl = {}
    for name, path in HOME_MODELS.items():
        try:
            with open(path, 'rb') as f:
                mdls[name] = pickle.load(f)
            expl[name] = shap.TreeExplainer(mdls[name])
        except FileNotFoundError:
            mdls[name] = None
            expl[name] = None
    return mdls, expl
dfs = load_data()
mdls, expl = load_models()# — Sidebar
with st.sidebar:
    st.header("⚙️ Controls")
    selected_home = st.selectbox("Select Home",
    list(HOME_DATA.keys()))
    st.divider()
    st.markdown("***About**")
    st.info(
        "EnergyGuard Mini demonstrates:\n"
        "- ML Forecasting\n"
        "- Anomaly Detection\n"
        "- Federated Learning\n"
        "- SHAP Explainability\n"
        "- Agentic AI")
    # — Get data for selected home
    df = dfs.get(selected_home)
    model = mdls.get(selected_home)
    if df is None:
        st.error(f"Data for {selected_home} not found. Run
        01_data_preparation.py first.")
    st.stop()
```

```

===== # SECTION 1: Overview Metrics#
=====st.subheader("📊 Overview
Metrics")col1, col2, col3, col4 = st.columns(4)with col1: st.metric("Avg
Consumption", f"{df['consumption_kwh'].mean():.2f} kWh")with col2:
st.metric("Max Consumption", f"{df['consumption_kwh'].max():.2f} kWh")with
col3: st.metric("Total Anomalies", f"{int(df['is_anomaly'].sum())}")with
col4: st.metric("Data Points", f"{len(df):,}")st.divider()#
===== # SECTION 2: Consumption Chart#
=====st.subheader("📈 Energy
Consumption Pattern")n_points = st.slider("Number of data points to show", 96,
500, 200)sample = df.tail(n_points).copy()sample =
sample.reset_index(drop=True)fig, ax = plt.subplots(figsize=(14,
4))ax.plot(sample.index, sample['consumption_kwh'], color='#2E75B6',
lw=1.2, label='Consumption')# Highlight anomaliesanomaly_pts =
sample[sample['is_anomaly'] == 1]if len(anomaly_pts) > 0:
ax.scatter(anomaly_pts.index, anomaly_pts['consumption_kwh'],
color='red', s=60, zorder=5, label='Anomaly (injected)')ax.set_xlabel('Time
step')ax.set_ylabel('kWh')ax.set_title(f'{selected_home} - Energy
Consumption')ax.legend()ax.grid(True,
alpha=0.3)plt.tight_layout()st.pyplot(fig)plt.close()st.divider()#
===== # SECTION 3: ML Forecasting#
=====st.subheader("🧠 ML
Forecasting")if model is None: st.warning("Model not found. Run
02_ml_forecasting.py first.")else: col1, col2 = st.columns([2, 1])
with col1: # Show actual vs predicted for test set split_idx =
int(len(df) * 0.8) X_test = df[FEATURES].iloc[split_idx:].values
y_test = df[TARGET].iloc[split_idx:].values y_pred =
model.predict(X_test) fig2, ax2 = plt.subplots(figsize=(10, 4))
show_n = 200 ax2.plot(y_test[-show_n:], label='Actual',
color='steelblue', lw=1.5) ax2.plot(y_pred[-show_n:], label='Predicted',
color='darkorange', lw=1.5, linestyle='--') ax2.set_xlabel('Time step')
ax2.set_ylabel('kWh') ax2.set_title(f'{selected_home} - Actual vs
Predicted') ax2.legend() ax2.grid(True, alpha=0.3)
plt.tight_layout() st.pyplot(fig2) plt.close() with col2:
mae = mean_absolute_error(y_test, y_pred) rmse =
np.sqrt(mean_squared_error(y_test, y_pred)) st.metric("Test
MAE", f"{mae:.4f} kWh") st.metric("Test RMSE", f"{rmse:.4f} kWh")
latest = df[FEATURES].iloc[-1].values.reshape(1, -1) next_pred =
model.predict(latest)[0] st.metric("Next Prediction", f"{next_pred:.3f}
kWh",
delta=f"{next_pred - df['consumption_kwh'].mean():+.3f}
vs avg")st.divider()# ===== # SECTION
4: Anomaly Detection#
=====st.subheader("🚨 Anomaly
Detection")X_anomaly = df[FEATURES].valuesiso =
IsolationForest(contamination=0.02, random_state=42,
n_estimators=100)iso.fit(X_anomaly)pred_labels =
iso.predict(X_anomaly)scores_raw = iso.score_samples(X_anomaly)scores_norm = 1
- ((scores_raw - scores_raw.min()) / (scores_raw.max() -
scores_raw.min() + 1e-6))df['anomaly_score'] =
scores_normdf['detected_anomaly'] = (pred_labels == -1).astype(int)col1, col2,
col3 = st.columns(3)current_status = "🔴 ANOMALY" if pred_labels[-1] == -1 else
"🟢 NORMAL"with col1: st.metric("Current Status", current_status)with col2:
st.metric("Current Anomaly Score", f"{scores_norm[-1]:.3f}")with col3:
st.metric("Detected Anomalies", f"{(pred_labels == -1).sum()}")fig3, ax3 =
plt.subplots(figsize=(14, 3))ax3.fill_between(range(len(scores_norm[-300:])),
scores_norm[-300:], alpha=0.6,
color='darkorange')ax3.axhline(y=0.7, color='red', linestyle='--',
label='Alert threshold (0.7)', lw=1.5)ax3.set_xlabel('Time
step')ax3.set_ylabel('Anomaly Score')ax3.set_title(f'{selected_home} - Anomaly
Scores (last 300 points)')ax3.legend()ax3.grid(True,
alpha=0.3)plt.tight_layout()st.pyplot(fig3)plt.close()st.divider()#
===== # SECTION 5: SHAP Explanation#
=====st.subheader("🔍 Explainable AI
(SHAP)")if model is None or selected_home not in explainers:
st.warning("Model not loaded.")else: explainer = explainers[selected_home]
X_sample = pd.DataFrame(df[FEATURES].head(300), columns=FEATURES) with
st.spinner("Calculating SHAP values..."): shap_values =
explainer.shap_values(X_sample) mean_shap =

```

```

np.abs(shap_values).mean(axis=0) importance = pd.DataFrame({
'Feature': FEATURES, 'Importance': mean_shap,
}).sort_values('Importance', ascending=True) coll, col2 = st.columns(2)
with coll: fig4, ax4 = plt.subplots(figsize=(8, 5))
ax4.barh(importance['Feature'], importance['Importance'],
color='#2E75B6', alpha=0.8) ax4.set_xlabel('Mean |SHAP value|')
ax4.set_title('Feature Importance (SHAP)') ax4.grid(True, alpha=0.3,
axis='x') plt.tight_layout() st.pyplot(fig4) plt.close()
with col2: # Single prediction explanation idx = 100
sample_shap = shap_values[idx] feat_vals = X_sample.iloc[idx]
contribs = pd.DataFrame({'Feature': FEATURES, 'SHAP':
sample_shap, 'Value': feat_vals.values,
}).sort_values('SHAP', key=abs, ascending=False).head(6) fig5,
ax5 = plt.subplots(figsize=(8, 5)) colors = ['forestgreen' if v > 0 else
'crimson' for v in contribs['SHAP']]
ax5.barh(contribs['Feature'], contribs['SHAP'], color=colors,
alpha=0.8) ax5.axvline(x=0, color='black', lw=0.8)
ax5.set_xlabel('SHAP value') ax5.set_title('Single Prediction
Explanation') ax5.grid(True, alpha=0.3, axis='x')
plt.tight_layout() st.pyplot(fig5) plt.close() st.divider()#
===== # SECTION 6: Federated Learning
Status# =====st.subheader("🌐
Federated Learning Status")fl_coll, fl_col2, fl_col3 = st.columns(3)for col,
(name, path) in zip([fl_coll, fl_col2, fl_col3], HOME_DATA.items()): with
col: try: df_h = pd.read_csv(path).dropna() m_h =
models.get(name) status = "✅ Ready" if m_h is not None else "⌚
Pending"
st.info(f"***{name}**\n\n{status}\n\nSamples:
{len(df_h):,}") except Exception: st.error(f"***{name}** -
Data missing")st.caption("✅ FedAvg complete - 5 rounds - Raw data remained
local")st.divider()# ===== # SECTION
7: AI Agent Recommendation#
=====st.subheader("🤖 AI Agent
Recommendation")if model: latest_features =
df[FEATURES].iloc[-1].values.reshape(1, -1) prediction =
model.predict(latest_features)[0] current_anomaly_score = scores_norm[-1]
is_anomaly = pred_labels[-1] == -1 if is_anomaly and prediction > 5.0:
severity = "CRITICAL" box_class = "alert-critical" icon =
"🔴" message = (f"Critical anomaly detected at {selected_home}! "
f"Predicted consumption {prediction:.2f} kWh is extremely high. "
"Inspect appliances immediately.") elif is_anomaly or prediction > 4.0:
severity = "WARNING" box_class = "alert-warning" icon = "🟡"
message = (f"Elevated consumption detected at {selected_home}. "
f"Predicted: {prediction:.2f} kWh. " "Review active
appliances.") else: severity = "NORMAL" box_class =
"alert-normal" icon = "🟢" message = (f"{selected_home} is
operating normally. " f"Predicted consumption:
{prediction:.2f} kWh. " "No action required.")
st.markdown(f'<div class="{box_class}">' f'<strong>{icon}>
{severity}</strong><br/>{message}' f'</div>',
unsafe_allow_html=True) col_pred, col_score = st.columns(2)
col_pred.metric("Next Predicted Consumption", f"{prediction:.3f} kWh")
col_score.metric("Anomaly Score", f"{current_anomaly_score:.3f}")

```

How to run:

```
streamlit run 09_dashboard.py# Opens automatically at http://localhost:8501
```

Part 11: Running the Complete Project

11.1 Complete Run Sequence

Run the scripts in this exact order. Each step builds on the previous one.

```
# Step 1: Generate all data (must run first)python 01_data_preparation.py# →
Creates homes/home_a.csv, homes/home_b.csv, homes/home_c.csv# Step 2: Train
forecasting modelspython 02_ml_forecasting.py# → Creates models/model_a.pkl,
model_b.pkl, model_c.pkl# Step 3: Run anomaly detectionpython
03_anomaly_detection.py# → Shows detection results, saves charts# Step 4:
Compare approachespython 04_virtual_homes.py# → Shows local vs centralized
comparison# Step 5: Federated Learningpython 05_federated_learning.py# → Runs 5
FL rounds, shows convergence# Step 6: SHAP explanationspython
06_shap_explainability.py# → Shows feature importance and waterfall charts#
Step 7: Agent demopython 07_energy_agent.py# → Demonstrates rule-based agent#
Step 8: Start the API (keep this running in background)python 08_api.py# →
Starts server at http://localhost:8000# Step 9: Start the dashboard (in a NEW
terminal tab)streamlit run 09_dashboard.py# → Opens at http://localhost:8501
```

11.2 What Success Looks Like

Script	Success indicator
01_data_preparation.py	3 CSV files in homes/ folder, chart showing consumption patterns
02_ml_forecasting.py	3 .pkl model files, MAE values printed, forecast charts
03_anomaly_detection.py	Precision/recall scores, anomaly detection charts
04_virtual_homes.py	Comparison table showing local vs centralized MAE
05_federated_learning.py	Convergence chart showing global MAE improving over 5 rounds
06_shap_explainability.py	Feature importance bar charts, waterfall plots
07_energy_agent.py	NORMAL and CRITICAL recommendations printed
08_api.py	Server running, /docs shows all endpoints
09_dashboard.py	Browser opens with interactive dashboard showing all sections

Part 12: Testing Your Project

12.1 Key Things to Verify

After building the project, verify these key properties to ensure everything is working correctly:

Verify Non-IID Data

```
import pandas as pd
df_a = pd.read_csv('homes/home_a.csv')
df_b = pd.read_csv('homes/home_b.csv')
df_c = pd.read_csv('homes/home_c.csv')
# These MUST be different - if they're the same, data isn't Non-IID
print("Home A mean:", df_a['consumption_kwh'].mean()) # ~1.5
print("Home B mean:", df_b['consumption_kwh'].mean()) # ~3.5
print("Home C mean:", df_c['consumption_kwh'].mean()) # ~2.0
```

Verify Federated Privacy

```
# In a real FL system, this check confirms raw data never left each home.
# In our simulation, verify that federated_learning.py never accesses
# another home's CSV during local training.
# Open 05_federated_learning.py and confirm:
# - FederatedClient only loads its own data
# - Only model.predict() outputs are shared, not raw data
```

Verify SHAP Explanations Sum to Prediction

```
# SHAP values must sum to: prediction - base_value
# If this doesn't hold, something is wrong with your SHAP setup
import numpy as np
predicted = 3.16
base_value = 1.8
shap_sum = 0.81 + 0.42 + 0.21 - 0.08 # sum of all SHAP values
print(f"Prediction: {predicted}")
print(f"Base + SHAP: {base_value + shap_sum:.2f}")
# Should be equal (or very close)
```

12.2 Common Errors and Fixes

Error	Cause	Fix
FileNotFoundError: homes/home_a.csv	Data not generated yet	Run 01_data_preparation.py first
FileNotFoundError: models/model_a.pkl	Model not trained	Run 02_ml_forecasting.py first
ModuleNotFoundError: shap	Library not installed	pip install shap
Address already in use (port 8000)	API already running	Kill the other process or use a different port
streamlit: command not found	Streamlit not installed	pip install streamlit
SHAP values don't sum to prediction	Using wrong explainer type	Use shap.TreeExplainer for Random Forest

12.3 Key Concepts to Explain in Your FYP Defence

After completing this project, you should be able to explain these concepts in your own words:

1. What is Federated Learning and why does it preserve privacy?

2. What is Non-IID data and why is it a challenge?
3. What is FedAvg and how does the aggregation step work?
4. What does a SHAP value represent for a single prediction?
5. What is the difference between supervised and unsupervised ML?
6. Why do we split data into training and testing sets?
7. What makes something an "AI agent" vs a simple function?
8. What is MAE and what does a value of 0.3 kWh mean?
9. Why is overfitting bad and how do you detect it?
10. How does a REST API connect an ML model to a dashboard?

EnergyGuard Mini — Complete Build Guide

This project teaches every core concept required for the GridGuard FYP. Build it once completely, understand every line, and the FYP architecture will make complete sense.